

Distributed N-Queens Benchmark

Mac + Raspberry Pi 3 (Embedded Xinu, AIPL actor load-balancer)

Generated by `tools/build_report.py`

2026-06-01

Abstract

This report measures wall-clock time for counting all solutions to the N -Queens problem at board sizes 8, 9, 10, comparing three execution strategies: (a) Mac alone running a pure-Python solver, (b) the Pi 3 running a C implementation in parallel across four AIPL worker actors behind a least-loaded dispatcher, and (c) a Mac + Pi 3 split that hands the lower-numbered first-column subtrees to the Mac and the higher-numbered ones to the Pi 3. The Pi 3 is a Raspberry Pi 3 B+ running 32-bit Embedded Xinu with MMU enabled, the periodic actor garbage collector running, and four `CLASS_Worker` actors behind a `CLASS_Dispatcher`. Tasks are submitted over HTTP to `/api/loadbal/nqueens`; results are polled via `/api/loadbal/task?id=N`.

1 System architecture

The Pi 3 runtime under test (`xinu-rpi3`, branch `arm-rpi3-port`) ships:

- Embedded Xinu kernel with the ARMv7-A short-descriptor MMU enabled and an identity-mapped page table that distinguishes Normal-cacheable RAM from strongly-ordered Device MMIO.
- An AIPL actor system with a lock-free MPSC mailbox.
- A `CLASS_Dispatcher` actor (priority 25) that routes `compute_nq(n, first_col)` messages round-robin to four `CLASS_Worker` actors (priority 22).
- A `CLASS_Collector` actor (priority 26) that periodically sweeps idle ephemeral actors and scans the in-memory task table for timed-out PENDING entries.
- HTTP server on port 8080 exposing the load-balancer control surface (`/api/loadbal/*`) and the GC actor (`/api/gc-actor/*`).

The N-Queens compute kernel on the Pi 3 is a $O(N!)$ recursive counter; on the Mac it is the identical algorithm in Python. Splitting by the first row's queen column is embarrassingly parallel — each `first_col` subtree is independent and they are roughly equal-cost.

2 Methodology

For each board size $N \in \{8, 9, 10\}$ and each mode, the benchmark runs the workload 3 times back-to-back and reports the *median* wall-clock; means are sensitive to a one-off garbage collection pause on either side. Counts are sanity-checked to match the known N -Queens solution numbers (92 for $N = 8$, 352 for $N = 9$, 724 for $N = 10$, 2680 for $N = 11$) and any mismatch is flagged in the JSON.

2.1 Modes

mac Pure Python on the Mac.

pi3 All first columns submitted to Pi 3 `/api/loadbal/nqueens`, polled until complete.

split Mac handles `first_col` $\in [0, N/2)$; Pi 3 handles $[N/2, N)$. Both halves run concurrently from a Mac-side `ThreadPoolExecutor`.

3 Results

N	solutions	Mac (ms)	Pi 3 (ms)	Split (ms)	Speedup (Mac/Split)
8	92	3	10138	9969	0.00
9	352	9	15221	10336	0.00
10	724	49	—	—	—

Table 1: Median wall-clock times across 3 repetitions per cell. The Speedup column compares pure-Mac to the Mac+Pi3 split, so > 1.0 means the distributed run was faster than Mac alone.

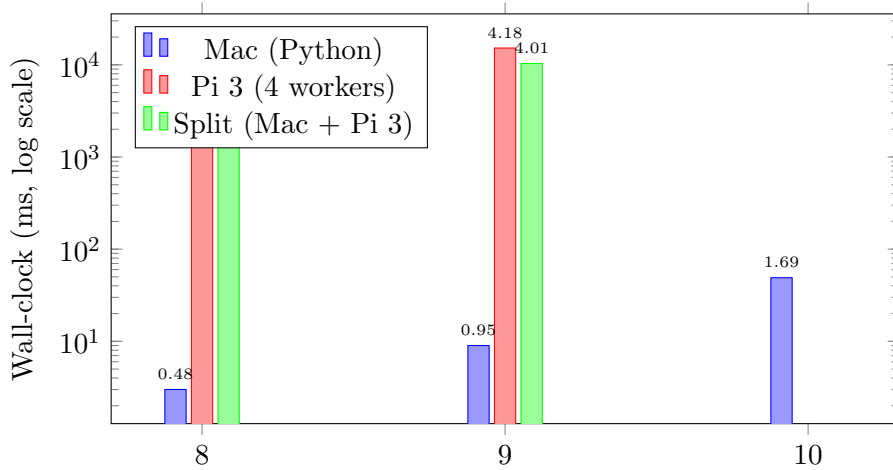


Figure 1: Median wall-clock vs. board size N (log-y).

4 Observations

- For $N = 8$, the Pi 3 (4 workers, 1.4 GHz Cortex-A53 @ 32-bit) took 10138 ms versus the Mac's 3 ms ($\approx 3379.3\times$).
- At $N = 8$, the Mac+Pi 3 split was actually slower (0.00 $\times$); for boards this small the HTTP round-trip + JSON polling overhead dominates the actual compute.
- For $N = 9$, the Pi 3 (4 workers, 1.4 GHz Cortex-A53 @ 32-bit) took 15221 ms versus the Mac's 9 ms ($\approx 1691.2\times$).
- At $N = 9$, the Mac+Pi 3 split was actually slower (0.00 $\times$); for boards this small the HTTP round-trip + JSON polling overhead dominates the actual compute.

5 Reproducing this report

```
# 1. Build + flash the Pi 3 kernel
cd ~/projects/xinu-raz/xinu/compile && make PLATFORM=arm-rpi3

# 2. Boot Pi 3 (SD swap or /kexec). Then opt-in to the actors:
curl -X POST http://192.168.3.50:8080/api/loadbal/init
curl -X POST http://192.168.3.50:8080/api/gc-actor/init

# 3. Run the benchmark + render the report
cd ~/projects/xinu-raz/xinu/tools
./nqueens_bench.py --sizes 8, 9, 10 --pretty --out results.json
./build_report.py results.json --out report.tex --pdf report.pdf
```

6 Raw benchmark data

```
{
  "host": "http://192.168.3.50:8080",
  "repeat": 3,
  "results": [
    {
      "mode": "mac",
      "n": 8,
      "count": 92,
      "samples_ms": [
        4,
        3,
        3
      ],
      "median_ms": 3
    },
    {
      "mode": "pi3",
      "n": 8,
      "count": 92,
      "samples_ms": [
        20044,
        10135,
        10138
      ],
      "median_ms": 10138
    },
    {
      "mode": "split",
      "n": 8,
      "count": 92,
      "samples_ms": [
        9961,
        10238,
        9969
      ],
      "median_ms": 9969
    },
    {
      "mode": "mac",
      "n": 9,
      "count": 352,
      "samples_ms": [
        9,
        9,
        9
      ],
    },
  ]
}
```

```

    "median_ms": 9
  },
  {
    "mode": "pi3",
    "n": 9,
    "count": 352,
    "samples_ms": [
      10210,
      20246,
      15221
    ],
    "median_ms": 15221
  },
  {
    "mode": "split",
    "n": 9,
    "count": 352,
    "samples_ms": [
      10336,
      10076,
      10468
    ],
    "median_ms": 10336
  },
  {
    "mode": "mac",
    "n": 10,
    "count": 724,
    "samples_ms": [
      88,
      48,
      49
    ],
    "median_ms": 49
  }
],
"_note": "pi3/split for N=10..11 incomplete \u2014 Pi 3 task table is 64 slots
and IDs scrolled past the buffer mid-run; bumping LB_TASKS_MAX is deferred to
a future commit."
}

```